

PRÁCTICA 2 - SPARK STRUCTURED STREAMING

Grupo: 6

Autores:

- **Diego Rodríguez Monteagudo,**
- **Alonso Rescalvo Casas**
- **Adrián Gutiérrez Toledo**

2. CARGA DE DATOS EN APACHE KAFKA

En este apartado preparamos y cargamos los datos de ejemplo en una cola (topic) de Apache Kafka, disponible en el mismo servidor empleado para la Fase 1 de la práctica.

Vamos a emplear los ficheros kafka-producer-confluent.py y kafka-consumer-confluent.py. Estos dos ficheros se deben ejecutar en un terminal (independiente o dentro de VS code) que tenga activado el entorno virtual kafka utilizado en la fase anterior de la práctica. Este entorno tiene instalado el paquete confluent_kafka que necesitan ambos clientes para funcionar.

Apartado 1

```
(kafka_env) alonso@Alonso: /mnt/c/Users/serli/Desktop/3ºGCID/SEGUNDO_CUATRI/SDPD2/practica2/kafka$ python kafka-producer-confluent.py
Produced event to topic purchases: key = jsmith-2      value = gift card
Produced event to topic purchases: key = eabara-2      value = t-shirts
Produced event to topic purchases: key = awalther-2    value = gift card
Produced event to topic purchases: key = jbernard-2    value = alarm clock
Produced event to topic purchases: key = sgarcia-2     value = alarm clock
Produced event to topic purchases: key = eabara-2      value = batteries
Produced event to topic purchases: key = htanaka-2     value = batteries
Produced event to topic purchases: key = eabara-2      value = gift card
Produced event to topic purchases: key = jsmith-2      value = t-shirts
Produced event to topic purchases: key = jsmith-2     value = gift card
```

Para comprobar la carga de datos en Kafka, se ejecutó primero el script kafka_producer_confluent.py, configurado con la dirección y el puerto del broker correspondiente. Este script generó varias líneas de productos aleatorias, con un mensaje que sigue un esquema de aviso de evento producido (Produced event to topic purchases), clave (key) y valor (value).

Apartado 2

```
(kafka_env) alonso@Alonso:/mnt/c/Users/ser12/Desktop/3*GCID/SEGUNDO_CUATRI/SDPD2/practica2/kafka$ python kafka-consumer-confluent.py
[]
Waiting...
Waiting...
Waiting...
Waiting...
Waiting...
Consumed event from topic purchases: key = jsmith-2      value = gift card
Consumed event from topic purchases: key = eabara-2      value = t-shirts
Consumed event from topic purchases: key = awalther-2    value = gift card
Consumed event from topic purchases: key = jbernard-2    value = alarm clock
Consumed event from topic purchases: key = sgarcia-2     value = alarm clock
Consumed event from topic purchases: key = eabara-2     value = batteries
Consumed event from topic purchases: key = htanaka-2     value = batteries
Consumed event from topic purchases: key = eabara-2     value = gift card
Consumed event from topic purchases: key = jsmith-2     value = t-shirts
Consumed event from topic purchases: key = jsmith-2     value = gift card
Waiting...
Waiting...
Waiting...
Waiting...
Waiting...
```

A continuación, se ejecutó `kafka_consumer_confluent.py`, configurado contra el mismo broker y el mismo topic en el que se hizo en `kafka_producer_confluent.py`. La salida obtenida muestra que los mensajes producidos previamente fueron consumidos correctamente, con un mensaje que sigue un esquema de aviso de evento consumido (Consumed event from topic purchases), clave (key) y valor (value).

3. KAFKA + SPARK STRUCTURED STREAMING

3.1 Instalación de PySpark

Crea un entorno virtual con Astral uv, con identificador `pyspark-411` y que utilice Python v3.11 o superior.

```
alonso@Alonso:/mnt/c/Users/ser12/Desktop/3*GCID/SEGUNDO_CUATRI/SDPD2/practica2$ uv venv pyspark-411 --python 3.12
Using CPython 3.12.3 interpreter at: /usr/bin/python3.12
Creating virtual environment at: pyspark-411
Activate with: source pyspark-411/bin/activate
```

Dentro del directorio del entorno virtual que acabas de crear, instala la versión 4.1.1 del paquete PySpark.

```
alonso@Alonso:/mnt/c/Users/ser12/Desktop/3*GCID/SEGUNDO_CUATRI/SDPD2/practica2$ source pyspark-411/bin/activate
(pyspark-411) alonso@Alonso:/mnt/c/Users/ser12/Desktop/3*GCID/SEGUNDO_CUATRI/SDPD2/practica2$ uv pip install pyspark[connect]==4.1.1
Using Python 3.12.3 environment at: pyspark-411
Resolved 13 packages in 4m 50s
Built pyspark==4.1.1
Prepared 13 packages in 51.80s
[0/13] Installing wheels...
warning: Failed to hardlink files; falling back to full copy. This may lead to degraded performance.
If the cache and target directories are on different filesystems, hardlinking may not be supported.
If this is intentional, set 'export UV_LINK_MODE=copy' or use '--link-mode=copy' to suppress this warning.
Installed 13 packages in 3m 59s
+ googleapis-common-protos==1.74.0
+ grpcio==1.80.0
+ grpcio-status==1.80.0
+ numpy==2.4.4
+ pandas==3.0.2
+ protobuf==6.33.6
+ py4j==0.10.9.9
+ pyarrow==24.0.0
+ pyspark==4.1.1
+ python-dateutil==2.9.0.post0
+ six==1.17.0
+ typing-extensions==4.15.0
+ zstandard==0.25.0
(pyspark-411) alonso@Alonso:/mnt/c/Users/ser12/Desktop/3*GCID/SEGUNDO_CUATRI/SDPD2/practica2$ |
```

Verifica que en subdirectorio `.venv/bin` han aparecido los programas binarios de Spark tras la instalación (`spark-shell`, `spark-submit`, etc.).

```
(pyspark-411) alonso@Alonso:/mnt/c/Users/ser12/Desktop/3°GCID/SEGUNDO_CUATRI/SDPD2/practica2/kafka$ ls $VIRTUAL_ENV/bin | grep spark
find-spark-home
find-spark-home.cmd
find_spark_home.py
load-spark-env.cmd
load-spark-env.sh
pyspark
pyspark.cmd
pyspark2.cmd
spark-class
spark-class.cmd
spark-class2.cmd
spark-connect-shell
spark-pipelines
spark-shell
spark-shell.cmd
spark-shell2.cmd
spark-sql
spark-sql.cmd
spark-sql2.cmd
spark-submit
spark-submit.cmd
spark-submit2.cmd
sparkR
sparkR.cmd
sparkR2.cmd
```

3.2 Consumidor Kafka desde Spark Structured Streaming

Iniciamos un contenedor de Docker y ejecutamos el script `struct_kafka_consumer_local.py` del cual previamente hemos modificado las versiones mediante la opción `kafka.bootstrap.servers`, que apunta al broker utilizado, para que coincida con las versiones instaladas en el ordenador en el que realizamos la ejecución.

```
(pyspark-411) alonso@Alonso:/mnt/c/Users/ser12/Desktop/3°GCID/SEGUNDO_CUATRI/SDPD2/practica2$ cd kafka/
(pyspark-411) alonso@Alonso:/mnt/c/Users/ser12/Desktop/3°GCID/SEGUNDO_CUATRI/SDPD2/practica2/kafka$ sudo docker compose
up -d
WARN[0000] /mnt/c/Users/ser12/Desktop/3°GCID/SEGUNDO_CUATRI/SDPD2/practica2/kafka/docker-compose.yml: the attribute 'version'
is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 1/1
✔ Container broker Started 0.4s
(pyspark-411) alonso@Alonso:/mnt/c/Users/ser12/Desktop/3°GCID/SEGUNDO_CUATRI/SDPD2/practica2/kafka$ sudo docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
c2853b444d59   apache/kafka:latest   "/__cacert_entrypoin..." 23 hours ago   Up 18 seconds   0.0.0.0:9092->9092/tcp, [::
]:9092->9092/tcp   broker
(pyspark-411) alonso@Alonso:/mnt/c/Users/ser12/Desktop/3°GCID/SEGUNDO_CUATRI/SDPD2/practica2/kafka$ python struct_kafka_
consumer_local.py
```

Los mensajes de log que demuestran que se han cargado las bibliotecas de dependencias adecuadas para construir el cliente de lectura de datos desde Kafka se pueden ver en la siguiente captura.

```
(pyspark-411) alonso@Alonso:/mnt/c/Users/ser12/Desktop/3°GCID/SEGUNDO_CUATRI/SDPD2/practica2/kafka$ python struct_kafka_consumer_local.py
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
26/06/20 18:35:11 WARN Utils: Your hostname, Alonso, resolves to a loopback address: 127.0.0.1; using 10.255.255.254 instead (on interface lo)
26/06/20 18:35:11 WARN Utils: Set sparkLocalMode.ip if you need to bind to another address
:: loading settings :: url = jar:file:/mnt/c/Users/ser12/Desktop/3°GCID/SEGUNDO_CUATRI/SDPD2/practica2/pyspark-411/lib/python3.12/site-packages/pyspark/jars/ivy-2.5.3-jar!/org/apache/ivy/core/settings/ivy
settings.xml
Ivy Default Cache set to: /home/alonso/.ivy2.5.2/cache
The jars for the packages stored in: /home/alonso/.ivy2.5.2/jars
org.apache.spark:spark-sql-kafka-0-10-2.13 added as a dependency
:: resolving dependencies :: org.apache.spark:spark-submit-parent-5ef61ecd-c8e7-488c-993a-ff102618dc42;1.0
  confs: [default]
    found org.apache.spark:spark-sql-kafka-0-10-2.13;4.1.1 in central
    found org.apache.spark:spark-token-provider-kafka-0-10-2.13;4.1.1 in central
    found org.apache.kafka:kafka-clients;3.9.1 in central
    found org.lz4:lz4-java;1.8.0 in central
    found org.xerial.snappy:snappy-java;1.1.10.8 in central
    found org.slf4j:slf4j-api;2.0.17 in central
    found org.apache.hadoop:hadoop-client-runtime;3.4.2 in central
    found org.apache.hadoop:hadoop-client-api;3.4.2 in central
    found com.google.code.findbugs:jsr305;3.0.0 in central
    found org.scala-lang.modules:scala-parallel-collections;2.13;1.2.0 in central
    found org.apache.commons:commons-pool2;2.12.1 in central
  :: resolution report :: resolve 879ms :: artifacts dl 17ms
  :: modules in use:
    com.google.code.findbugs:jsr305;3.0.0 from central in [default]
    org.apache.commons:commons-pool2;2.12.1 from central in [default]
    org.apache.hadoop:hadoop-client-api;3.4.2 from central in [default]
    org.apache.hadoop:hadoop-client-runtime;3.4.2 from central in [default]
    org.apache.kafka:kafka-clients;3.9.1 from central in [default]
    org.apache.spark:spark-token-provider-kafka-0-10-2.13;4.1.1 from central in [default]
    org.lz4:lz4-java;1.8.0 from central in [default]
    org.scala-lang.modules:scala-parallel-collections;2.13;1.2.0 from central in [default]
    org.slf4j:slf4j-api;2.0.17 from central in [default]
    org.xerial.snappy:snappy-java;1.1.10.8 from central in [default]
  |-----|
  | conf | number | search | dnmtd | evicted | number | artifacts |
  |-----|
  | default | 11 | 0 | 0 | 0 | 11 | 0 |
  :: retrieving :: org.apache.spark:spark-submit-parent-5ef61ecd-c8e7-488c-993a-ff102618dc42
    confs: [default]
    0 artifacts copied, 11 already retrieved (0KB/12ms)
26/06/20 18:35:16 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/06/20 18:35:17 WARN ResolveWriteToStream: Temporary checkpoint location created which is deleted normally when the query didn't fail: /tmp/temporary-cw0819ff-7d8a-4ced-8b52-5bb5451f7290. If it's required t
o delete it under any circumstances, please set spark.sql.streaming.forceDeleteTempCheckpointLocation to true. Important to know deleting temp checkpoint folder is best effort.
26/06/20 18:35:17 WARN ResolveWriteToStream: spark.sql.adaptive.enabled is not supported in streaming DataFrames/Datasets and will be disabled.
```

1. resolving dependencies:

Esta línea indica que Spark está resolviendo las dependencias externas que necesita para ejecutar el programa. En este caso, al utilizar Kafka como fuente de

entrada, Spark necesita cargar librerías adicionales que no forman parte del núcleo básico de PySpark.

2. found org.apache...:

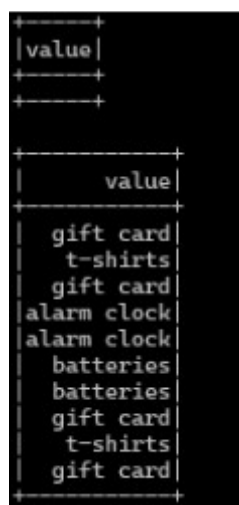
Este mensaje indica que Spark ha localizado correctamente los paquetes necesarios en el repositorio de dependencias. Entre ellos se encuentra el conector que permite integrar Spark Structured Streaming con Kafka, que es compatible con la versión de Spark utilizada en la práctica.

3. modules in use:

Esta parte muestra el conjunto de módulos que finalmente se han cargado para la ejecución. Además del conector principal de Kafka, aparecen otras librerías auxiliares necesarias para el funcionamiento del cliente, como dependencias relacionadas con compresión, comunicación y procesamiento interno.

En resumen, estos mensajes confirman que Spark ha incorporado correctamente las bibliotecas necesarias para conectarse a Kafka y consumir datos desde el topic configurado sin que se produzca ningún tipo de error. Por tanto, esta salida demuestra que el entorno está preparado para ejecutar el consumidor de Kafka mediante Spark Structured Streaming.

Esta es una de las tablas de salida que imprime el programa como resultado.



value
gift card
t-shirts
gift card
alarm clock
alarm clock
batteries
batteries
gift card
t-shirts
gift card

3.2.1 Preguntas

1. ¿Por que pensáis que no se obtiene ningún valor en la primera iteración que imprime por la terminal el resultado de la consulta a la tabla raw_data ?

No se obtiene ningún valor en la primera iteración porque el método `.start()` lanza la consulta de *streaming* de forma asíncrona. El bucle principal ejecuta el primer `.show()` inmediatamente, antes de que a Spark le haya dado tiempo a conectarse a Kafka, extraer los primeros mensajes y escribirlos en la tabla en

memoria `raw_data`. Tras el primer `sleep(3)`, el flujo de fondo ya ha tenido tiempo de volcar los datos, por lo que las siguientes iteraciones sí muestran los valores.

2. ¿Qué función tiene la siguiente línea de código para configurar el input stream de Kafka?

```
.option("startingOffsets", "earliest")
```

La función de esta opción es indicarle al consumidor de Spark que, al conectarse al topic de Kafka por primera vez, no debe esperar únicamente a los mensajes nuevos, sino que debe comenzar a leer desde el offset más antiguo disponible (el principio del stream). Esto permite recuperar y procesar todo el histórico de mensajes que ya estaban almacenados en Kafka antes de que se iniciara la consulta.

3. Busca información en la guía de programación de Spark Structured Streaming y las referencias de bibliografía para explicar qué tipo de stream de salida crea el siguiente fragmento de código. ¿Por qué podemos hacer consultas directamente sobre dicho stream, como si fuese una tabla de datos ordinaria?

```
describe_query = (input_data.writeStream.queryName("raw_data")
                  .format("memory").outputMode("append")
                  .start())
```

Tipo de Stream: Se trata de un Memory Sink. Es un receptor de datos que almacena el resultado del flujo estructurado en la memoria del *Driver* de Spark como una tabla interactiva.

Consulta como tabla ordinaria: Podemos realizar consultas SQL sobre él gracias a la opción `.queryName("raw_data")`. Esta instrucción registra el flujo de salida en el catálogo de Spark SQL con ese nombre específico, permitiendo que el motor de Spark trate la memoria donde se vuelcan los datos como una tabla relacional estándar.

Funcionamiento de append: En este modo de salida, en cada intervalo de activación (trigger), solo las filas nuevas procesadas en el último micro-lote se añaden a la tabla `raw_data`. Al realizar un `SELECT *`, visualizamos el histórico acumulado de todos los mensajes recibidos desde el inicio de la ejecución.

4. Implementa dos consultas sobre los datos obtenidos del stream consumido desde la cola de Kafka, utilizando dos modos de salida diferentes de Spark Structured Streaming y volcando la salida a un fichero de texto con

el nombre salida[N].txt, donde [N] se sustituye por el identificador de la salida correspondiente.

Para realizar este apartado se ha creado un archivo .py llamado consultas_finales.py.

En él se han implementado dos consultas sobre el flujo de datos consumido desde el topic purchases de Kafka. Para ello, primero se crea una sesión de Spark configurada con el conector spark-sql-kafka-0-10_2.13:4.1.1, necesario para que Spark Structured Streaming pueda leer datos desde Kafka. A continuación, se define un DataFrame de streaming que se conecta al broker Kafka en localhost:9092, se suscribe al topic purchases y comienza la lectura desde el offset más antiguo disponible mediante la opción startingOffsets = earliest. Después, los valores recibidos desde Kafka se transforman de formato binario a texto mediante CAST(value AS STRING), obteniendo una columna llamada value con el contenido de cada mensaje.

La primera consulta utiliza el modo de salida append. En este caso, el objetivo es guardar los mensajes en crudo tal y como van llegando desde Kafka. Para ello, se utiliza foreachBatch, que permite procesar cada micro-lote generado por Spark y escribir sus filas en el fichero salida1.txt. El fichero se abre en modo append, por lo que cada nuevo mensaje recibido se añade al final sin borrar los anteriores. Esta consulta permite comprobar directamente qué datos está consumiendo Spark desde Kafka.

La segunda consulta utiliza el modo de salida complete. En este caso, primero se agrupan los mensajes por producto mediante groupBy("value") y se calcula el número total de apariciones de cada producto con count(). El resultado representa un resumen acumulado de ventas por producto. Al usar el modo complete, Spark recalcula y escribe en cada micro-lote la tabla completa de resultados agregados. Por este motivo, el fichero salida2.txt se abre en modo escritura, sobrescribiendo su contenido anterior para mantener siempre una versión actualizada del total.